

Advanced Algorithms and Data Structures

Lecture# 09: Greedy Algorithms

Greedy Algorithms

- A **greedy algorithm** is a type of optimization algorithm that makes locally optimal choices at each step to find a globally optimal solution.
- It operates on the principle of “taking the best option now” without considering the long-term consequences.
- Similar to dynamic programming, but simpler approach. It is also used for optimization problems.
- Greedy algorithms don’t always yield an optimal solution
- Makes the choice that looks best at the moment in order to get optimal solution.

Greedy Algorithms

- **Define the problem:** Clearly state the problem to be solved and the objective to be optimized.
- **Identify the greedy choice:** Determine the locally optimal choice at each step based on the current state.
- **Make the greedy choice:** Select the greedy choice and update the current state.
- **Repeat:** Continue making greedy choices until a solution is reached.

Fractional Knapsack Problem

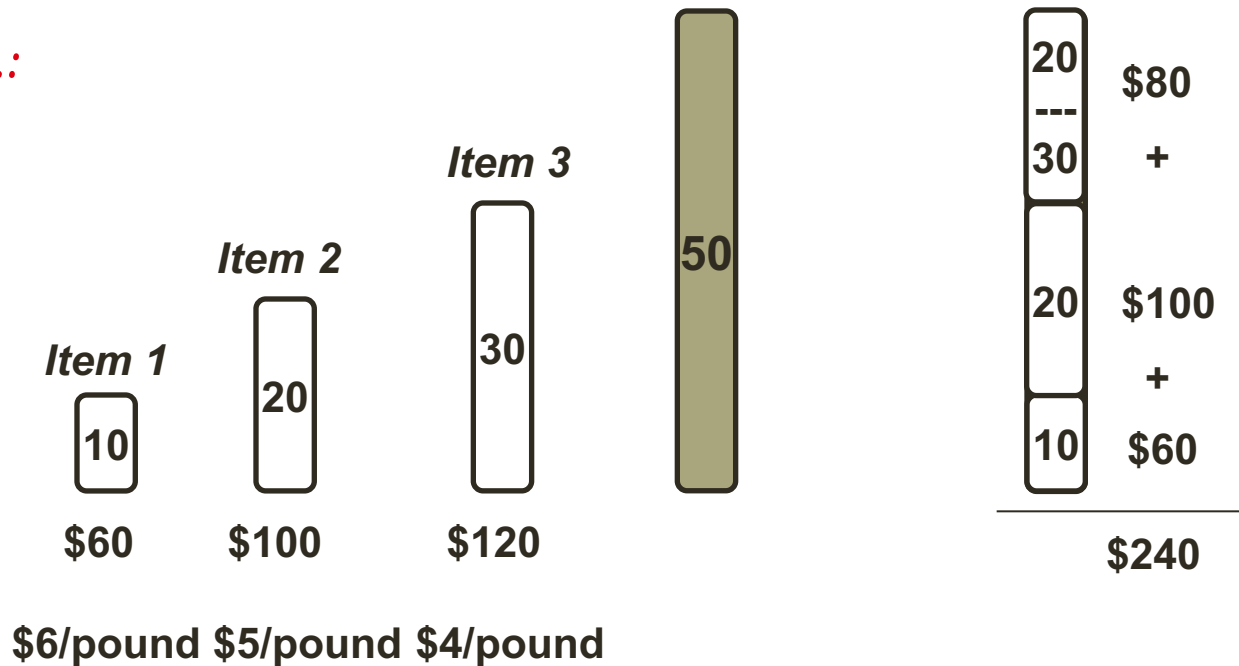
- Knapsack capacity: W
- There are n items: the i -th item has value v_i and weight w_i
- Goal:
 - find x_i such that for all $0 \leq x_i \leq 1, \quad i = 1, 2, \dots, n$

$$\sum w_i x_i \leq W \text{ and}$$

$$\sum x_i v_i \text{ is maximum}$$

Fractional Knapsack - Example

- E.g.:*



Fractional Knapsack Problem

- Greedy strategy 1:
 - Pick the item with the maximum value
- *E.g.:*
 - $W = 1$
 - $w_1 = 100, v_1 = 2$
 - $w_2 = 1, v_2 = 1$
 - Taking from the item with the maximum value:
Total value taken = $v_1/w_1 = 2/100$
 - Smaller than what the thief can take if choosing the other item
Total value (choose item 2) = $v_2/w_2 = 1$

Fractional Knapsack Problem

Greedy strategy 2:

- Pick the item with the maximum value per pound v_i/w_i
- If the supply of that element is exhausted and the thief can carry more: **take as much as possible from the item** with the next greatest value per pound
- It is good to order items based on their value per pound

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \dots \geq \frac{v_n}{w_n}$$

Fractional Knapsack Problem

Alg.: Fractional-Knapsack ($W, v[n], w[n]$)

1. While $w > 0$ and as long as there are items remaining
2. pick item with maximum v_i/w_i
3. $x_i \leftarrow \min(1, w/w_i)$
4. remove item i from list
5. $w \leftarrow w - x_i w_i$

- w – the amount of space remaining in the knapsack ($w = W$)
- Running time: $\Theta(n)$ if items already ordered; else $\Theta(n \lg n)$

Huffman Code Problem

- Huffman's algorithm achieves data compression by finding the best variable length binary encoding scheme for the symbols that occur in the file to be compressed.

Huffman Code Problem

- The more frequent a symbol occurs, the shorter should be the Huffman binary word representing it.
- The Huffman code is a prefix-free code.
 - No prefix of a code word is equal to another codeword.

Overview

- Huffman codes: compressing data (savings of 20% to 90%)
- Huffman's greedy algorithm uses a table of the frequencies of occurrence of each character to build up an optimal way of representing each character as a binary string

	a	b	c	d	e	f	← C: Alphabet
Frequency (in thousands)	45	13	12	16	9	5	
Fixed-length codeword	000	001	010	011	100	101	
Variable-length codeword	0	101	100	111	1101	1100	

Figure 16.3 A character-coding problem. A data file of 100,000 characters contains only the characters a–f, with the frequencies indicated. If each character is assigned a 3-bit codeword, the file can be encoded in 300,000 bits. Using the variable-length code shown, the file can be encoded in 224,000 bits.

Example

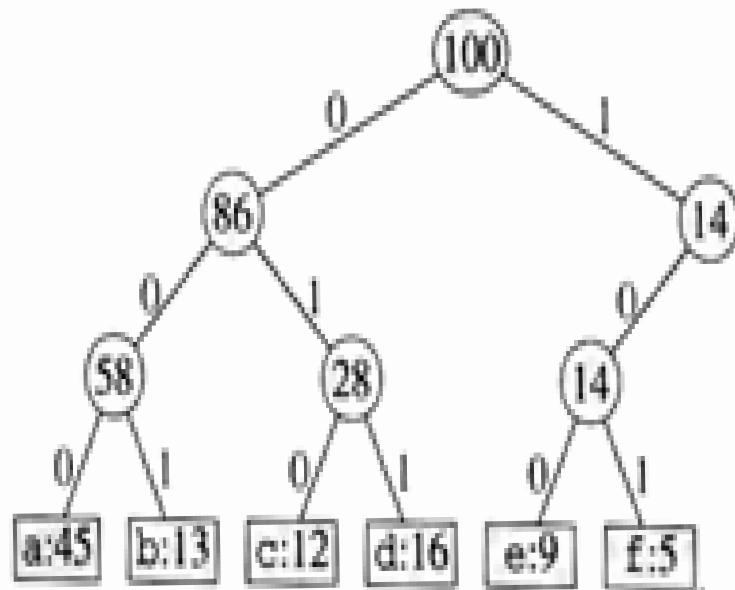
- Assume we are given a data file that contains only 6 symbols, namely a, b, c, d, e, f With the following frequency table:

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

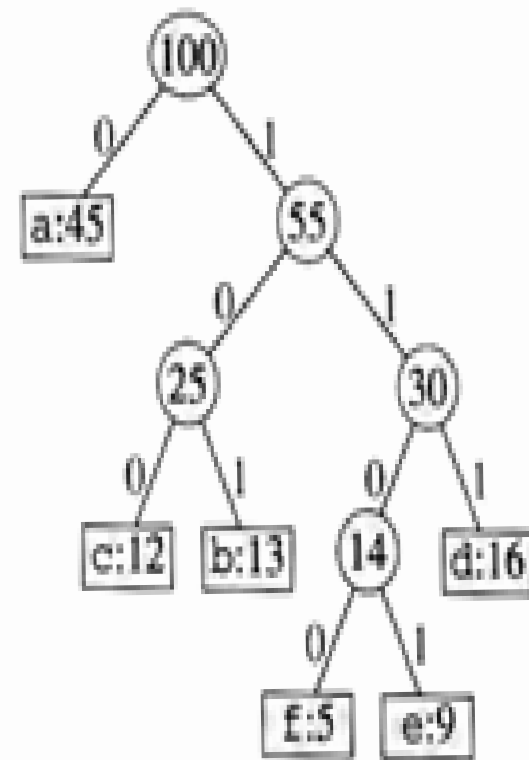
- Find a variable length prefix-free encoding scheme that compresses this data file as much as possible?

Huffman Code Problem

- Left tree represents a fixed length encoding scheme
- Right tree represents a Huffman encoding scheme



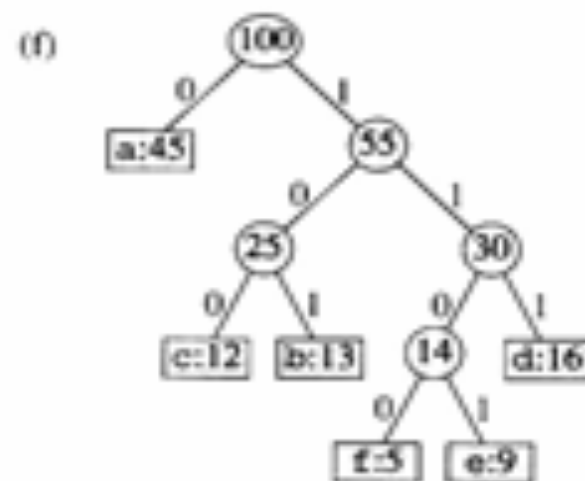
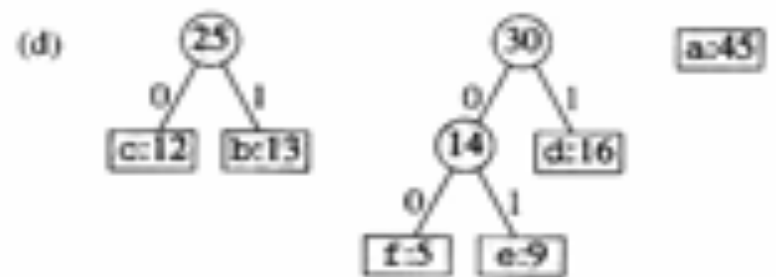
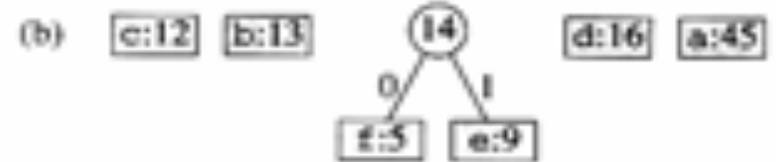
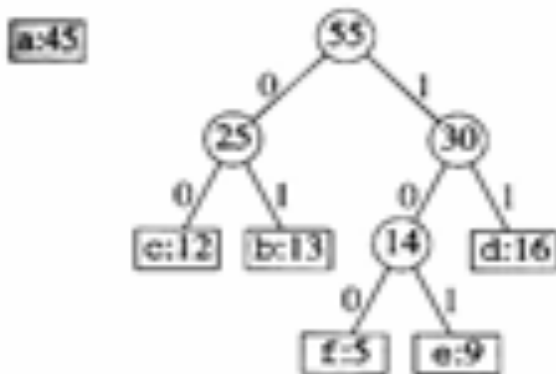
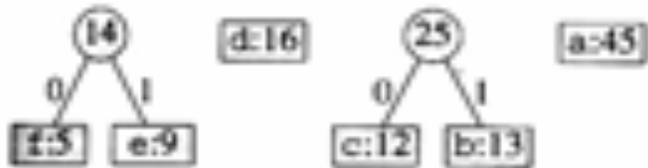
(a)



(b)

Example

f:5 e:9 c:12 b:13 d:16 a:45



Constructing A Huffman Code

HUFFMAN(C) // C is a set of n characters

1 $n \leftarrow |C|$

2 $Q \leftarrow C$ // Q is implemented as a binary min-heap $O(n)$

3 for $i \leftarrow 1$ to $n - 1$

Total computation time = $O(n \lg n)$

4 do allocate a new node z

5 $left[z] \leftarrow x \leftarrow \text{EXTRACT-MIN}(Q)$ $O(\lg n)$

6 $right[z] \leftarrow y \leftarrow \text{EXTRACT-MIN}(Q)$ $O(\lg n)$

7 $f[z] \leftarrow f[x] + f[y]$

8 $\text{INSERT}(Q, z)$ $O(\lg n)$

9 return $\text{EXTRACT-MIN}(Q)$

▷ Return the root of the tree.

Cost of a Tree T

- For each character c in the alphabet $\mathcal{C}$
 - let $f(c)$ be the frequency of c in the file
 - let $d_T(c)$ be the depth of c in the tree
 - It is also the length of the codeword. Why?
- Let $B(T)$ be the number of bits required to encode the file (called the cost of T)

$$B(T) = \sum_{c \in \mathcal{C}} f(c) d_T(c)$$

Huffman Code Problem

In the pseudocode that follows:

- we assume that C is a set of n characters and that each character $c \in C$ is an object with a defined frequency $f[c]$.
- The algorithm builds the tree T corresponding to the optimal code
- A min-priority queue Q , is used to identify the two least-frequent objects to merge together.
- The result of the merger of two objects is a new object whose frequency is the sum of the frequencies of the two objects that were merged.

Running time of Huffman's algorithm

- The running time of Huffman's algorithm assumes that Q is implemented as a binary min-heap.
- For a set C of n characters, the initialization of Q in line 2 can be performed in $O(n)$ time using the BUILD-MINHEAP
- The **for** loop in lines 3-8 is executed exactly $n - 1$ times, and since each heap operation requires time $O(\lg n)$, the loop contributes $O(n \lg n)$ to the running time. Thus, the total running time of HUFFMAN on a set of n characters is $O(n \lg n)$.

Prefix Code

- Prefix(-free) code: no codeword is also a prefix of some other codewords (Un-ambiguous)
 - An optimal data compression achievable by a character code can always be achieved with a prefix code
 - Simplify the encoding (compression) and decoding
 - Encoding: abc $\rightarrow$ 0 . 101. 100 = 0101100
 - Decoding: 001011101 = 0. 0. 101. 1101 $\rightarrow$ aabe
 - Use binary tree to represent prefix codes for easy decoding
- An optimal code is always represented by a **full binary tree**, in which every non-leaf node has two children
 - $|C|$ leaves and $|C|-1$ internal nodes Cost:

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

The diagram shows the formula $B(T) = \sum_{c \in C} f(c) d_T(c)$. A callout box labeled "Frequency of c" points to the term $f(c)$. Another callout box labeled "Depth of c (length of the codeword)" points to the term $d_T(c)$.

Huffman Code

- Reduce size of data by 20%-90% in general
- If no characters occur more frequently than others, then no advantage over ASCII
- Encoding:
 - Given the characters and their frequencies, perform the algorithm and generate a code. Write the characters using the code
- Decoding:
 - Given the Huffman tree, figure out what each character is (possible because of prefix property)

Application on Huffman code

- Both the .mp3 and .jpg file formats use Huffman coding at one stage of the compression

DP vs. Greedy Algorithms

- Dynamic programming
 - We make a choice at each step
 - The choice depends on solutions to subproblems
 - Bottom up solution, from smaller to larger subproblems
- Greedy algorithm
 - Make the greedy choice and THEN
 - Solve the subproblem arising after the choice is made
 - The choice we make may depend on previous choices, but not on solutions to subproblems
 - Top down solution, problems decrease in size

Applications of Greedy Algorithm

- Assigning tasks to resources to minimize waiting time or maximize efficiency.
- Selecting the most valuable items to fit into a knapsack with limited capacity.
- Dividing an image into regions with similar characteristics.
- Reducing the size of data by removing redundant information.

Limitations

- Greedy algorithms may not always find the best possible solution.
- The order in which the elements are considered can significantly impact the outcome.
- Greedy algorithms focus on local optimizations and may miss better solutions that require considering a broader context.
- Greedy algorithms are not applicable to problems where the greedy choice does not lead to an optimal solution.

Comparison among Greedy, DnC and DP algorithms

- Greedy algorithm: Makes locally optimal choices at each step with the hope of finding a global optimum.
- Divide and conquer algorithm: Breaks down a problem into smaller subproblems, solves each subproblem recursively, and then combines the solutions to the subproblems to solve the original problem.
- Dynamic programming algorithm: Solves subproblems recursively and stores their solutions to avoid repeated calculations.

Comparison among Greedy, DnC and DP algorithms

- Greedy algorithm: Finds the best solution among a set of possible solutions.
- Divide and conquer algorithm: Solves a problem by dividing it into smaller subproblems, solving each subproblem independently, and then combining the solutions to the subproblems to solve the original problem.
- Dynamic programming algorithm: Solves a problem by breaking it down into smaller subproblems and solving each subproblem recursively.

Comparison among Greedy, DnC and DP algorithms

- Greedy algorithm: $O(n \log n)$ or $O(n)$ depending on the problem.
- Divide and conquer algorithm: $O(n \log n)$ or $O(n^2)$ depending on the problem.
- Dynamic programming algorithm: $O(n^2)$ or $O(n^3)$ depending on the problem

Comparison among Greedy, DnC and DP algorithms

- Greedy algorithm: May or may not provide the optimal solution.
- Divide and conquer algorithm: May or may not provide the optimal solution.
- Dynamic programming algorithm: Guarantees the optimal solution.